MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Faculty of Automation and Computer Science

Active Suspension

Project documentation — System Identification

Name: POP Raul
Specialization: Automation and Applied Informatics
Group: 30135

Introduction

This work aims to identify a second-order system with complex conjugate poles, using only the experimental data: an input vector and an output vector. They come from a Simulink model excited with a chirp signal, that is, a sinusoid whose frequency increases over time.

The response in the figure below has the typical look of a second-order system: at low frequencies the output follows the input, and as the frequency increases its amplitude decreases.

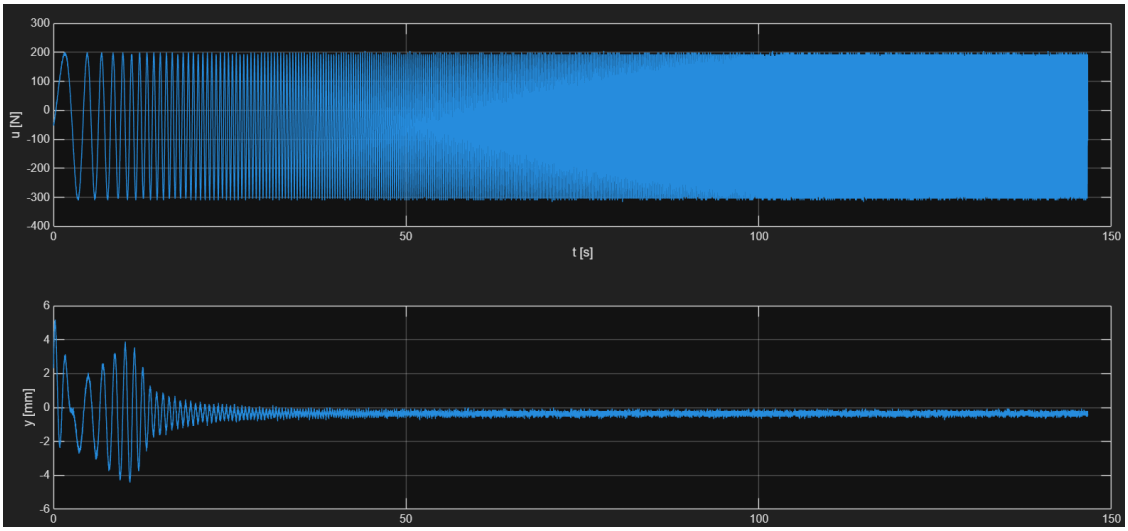


Figure 1 — Acquired signals: input u(t) and output y(t).

As a model, we choose the standard second-order transfer function with complex conjugate poles:

$$\mathrm{H}(s) = \frac{K{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

We therefore need to find three parameters: the proportionality constant K, the natural frequency ωn and the damping ratio ζ. For this we use the chirp signal method, described in the next chapter.

1. Chirp Signal Method

1.1 Chirp signal calibration

For the experiment to be useful, the chirp signal must clearly cover both the low and the medium frequencies, where the dynamics of the system is visible, as well as how the input frequencies are attenuated.

Of the whole experiment, only the frequency range [fmin, fmax] still needs calibration; the duration, the law and speed of frequency variation, and the sampling frequency were already set.

The range is chosen as [ωc/10, 10ωc], where ωc ≈ ωf, which led to the values:

```
fmin = 0.1/5;
fmax = 10/5;
Ain  = 385;
```

After calibration, the signals look as in the following figure:

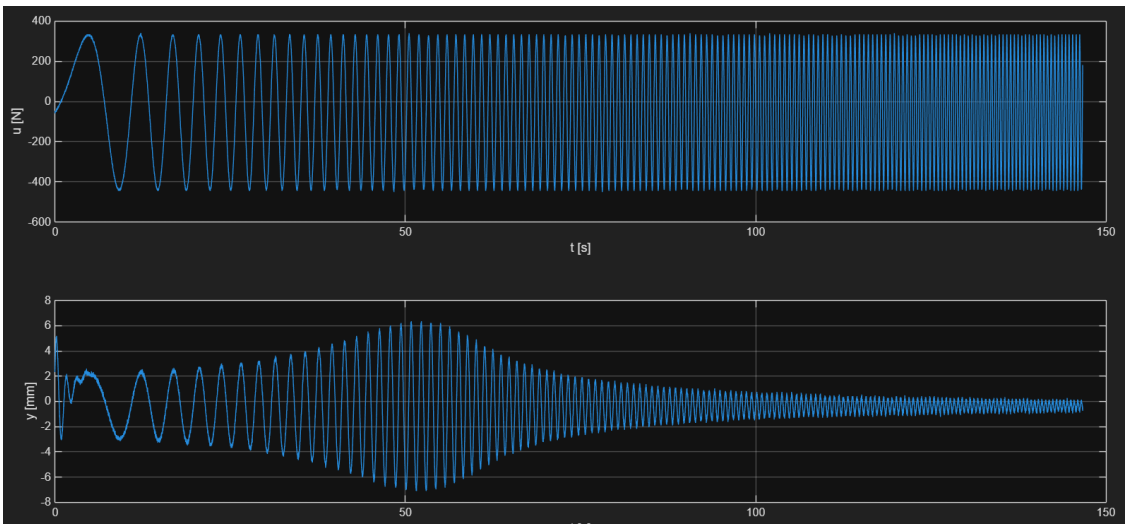


Figure 2 — Signals u(t) and y(t) after calibrating the frequency range.

1.2 Theoretical background

The response of the system to a chirp is easy to read on the plot. At low frequencies, the output passes proportionally with the constant K. Around ωr the amplitude increases and forms a “hump”, then decreases again. This is the resonance phenomenon, also visible on the Bode diagram, and the overall behavior is that of a low-pass filter: the input signal only changes its frequency, and the output amplitude changes accordingly. The existence of resonance requires ζ to lie in the interval (0, √2/2).

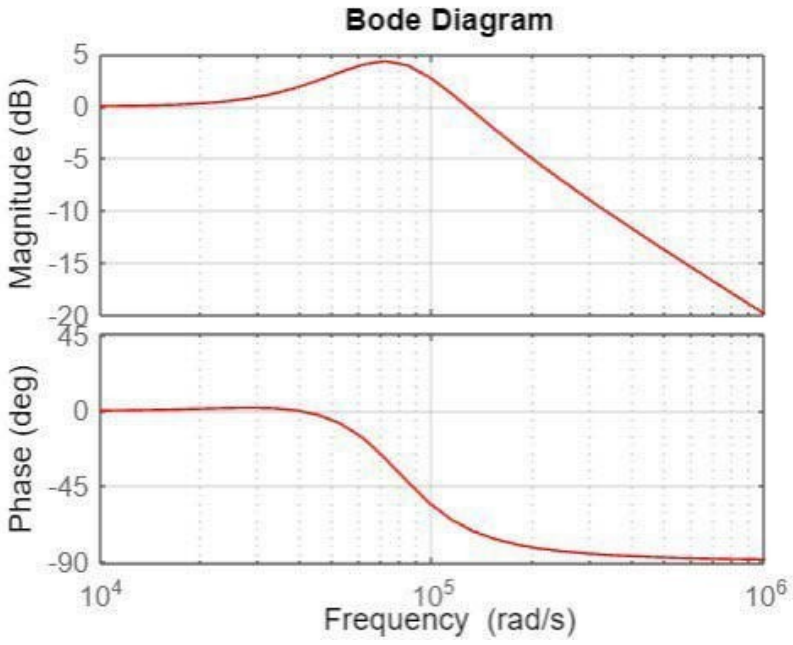


Figure 3 — Bode diagram (source: r/ControlTheory).

1.3 Determining the proportionality constant K

The constant K is obtained from the low-frequency behavior. The first maximum of the output is taken, after the effect of the initial conditions has vanished, and divided by the amplitude of the first maximum of the input:

$$K \approx \frac{A_y^{\mathrm{CFJ}}}{A_u^{\mathrm{CFJ}}}$$

In the code, these maxima are indexed by i1 (for y) and i2 (for u):

```
i1 = 6083; % first clear max for y
i2 = 6178; % first clear max for u

K = y(i1) / u(i2);
```

Alternatively, K could be computed as the ratio between the mean values of the input and output signals in steady state.

1.4 Determining the parameters ζ and ωn

Determining ζ and ωn starts from the resonance peak, that is, from the maximum gain of the output relative to that of the input:

$$M_r \approx \frac{A_y^r}{A_u^r}$$

The resonant frequency is obtained from the oscillation period, computed as the mean between the period to the left and the one to the right of the resonance peak:

$$\omega_{\text{osc}} = \frac{2\pi}{\text{T}_{\text{osc}}}$$

On the other hand, between ωr and the system parameters there is the relation:

$$\omega_r = \omega_n \sqrt{1 - 2\zeta^2}$$

while the value of the resonance peak, evaluated on the transfer function, is:

$$M_r = |\text{H}(j\omega_r)| = \frac{K}{2\zeta\sqrt{1-\zeta^2}}$$

From the last equality, by squaring and rearranging, a biquadratic equation in ζ results:

$$4M_r^2\zeta^4 - 4M_r^2\zeta^2 + K^2 = 0$$

whose solution, lying in the interval (0, √2/2), is:

$$\zeta = \sqrt{\frac{1 - \sqrt{1 - \left(\frac{K}{M_r}\right)^2}}{2}}$$

Since the resonant frequency can be approximated by the measured oscillation frequency:

$$\omega_r \approx \omega_{\text{osc}}$$

the natural frequency follows immediately from the relation between ωr and ωn:

$$\omega_n = \frac{\omega_r}{\sqrt{1 - 2\zeta^2}}$$

Concretely, the amplitudes at resonance are computed from the corresponding maximum and minimum of the output (i3, i4) and of the input (i5, i6), and the oscillation period from three successive peaks around resonance (i7, i8, i9):

```
i3 = 25421; % max rez y
i4 = 25782; % min rez y
i5 = 23800; % max u
i6 = 24187; % min u
```

```
Amply = (y(i3) - y(i4))/2;
Amplu = (u(i5) - u(i6))/2;

i7 = 24698; % first peak
i8 = 25383; % rez peak
i9 = 26140; % second peak

ToscPrev  = t(i8) - t(i7);
ToscAfter = t(i9) - t(i8);
Tosc = (ToscPrev + ToscAfter)/2;

wosc = 2*pi/Tosc;% =~ wrez
wr = wosc;
Mr = Amply / Amplu;

zeta = sqrt((1 - sqrt(1 - (K/Mr)^2) )/2);
wn = wr/ sqrt(1 - 2*zeta^2);
```

Running the code above gives the parameter values:

$$K = 0.0066, \quad \zeta = 0.2008, \quad \omega_n = 4.5443$$

and the identified transfer function, with numerical values, becomes:

$$\mathrm{H}(s) = \frac{0.1363}{s^2 + 1.825s + 20.65}$$

1.5 Model reconstruction

With the parameters K, ζ and ωn determined, the model is reconstructed in the two usual forms: the transfer function and the state-space representation.

Transfer function

The transfer function is built directly from the three parameters, and the response to the measured input is obtained with lsim:

```
H = tf(K*wn^2, [1, 2*zeta*wn, wn^2]);
y_sim_tf = lsim(H, u, t);
```

The simulation starts by default from zero initial conditions, so the estimated response needs some time to reach the real one. That is why the errors J and eps_MPN are larger than for the state-space representation (section 1.6).

State space

The same three values also define the state-space model. The state matrix is:

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -\omega_n{}^2 & -2\zeta\omega_n \end{bmatrix};$$

the input vector:

$$\mathrm{B} = \begin{bmatrix} 0 \\ K\omega_n{}^2 \end{bmatrix};$$

and the output is the first state, with no direct feedthrough from the input:

$$\mathrm{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}; \mathrm{D} = 0;$$

With the identified values, the matrices A and B become:

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -20.65 & -1.825 \end{bmatrix}$$

$$\mathrm{B} = \begin{bmatrix} 0 \\ 0.1363 \end{bmatrix}$$

The model is simulated again with lsim, this time with explicit initial conditions:

```
sys = ss(A, B, C, D);
y_sim_ss = lsim(sys, u, t, [y(1) 25]);
```

The initial position is known from the measured signal. The initial velocity can be found either analytically, by differentiating the signal and taking the first value, or numerically, by the interval halving (bisection) method, chosen here. Through successive calibrations the value 25 was reached, beyond which any other change worsened the validation error.

1.6 Model validation

The models are compared with the measured signal using two indicators, those presented in the System Identification course. The first is:

$$J = \frac{\| \varepsilon \|_2}{\sqrt{N}}$$

and the second, normalized with respect to the variation of the output around its mean:

$$\varepsilon_{\mathrm{MPN}} = \frac{\| \varepsilon \|_2}{\| y - \bar{y} \|_2}$$

To reduce the error even further, the measured output was filtered in two ways: with a 2nd-order Butterworth filter and with a moving average filter.

State-space model

The state-space model overlaps the measured signal very well, both raw and filtered. Butterworth filtering brings the error down to eps_MPN = 3.20%, and the moving average to eps_MPN = 2.82%, both below the imposed 10% threshold.

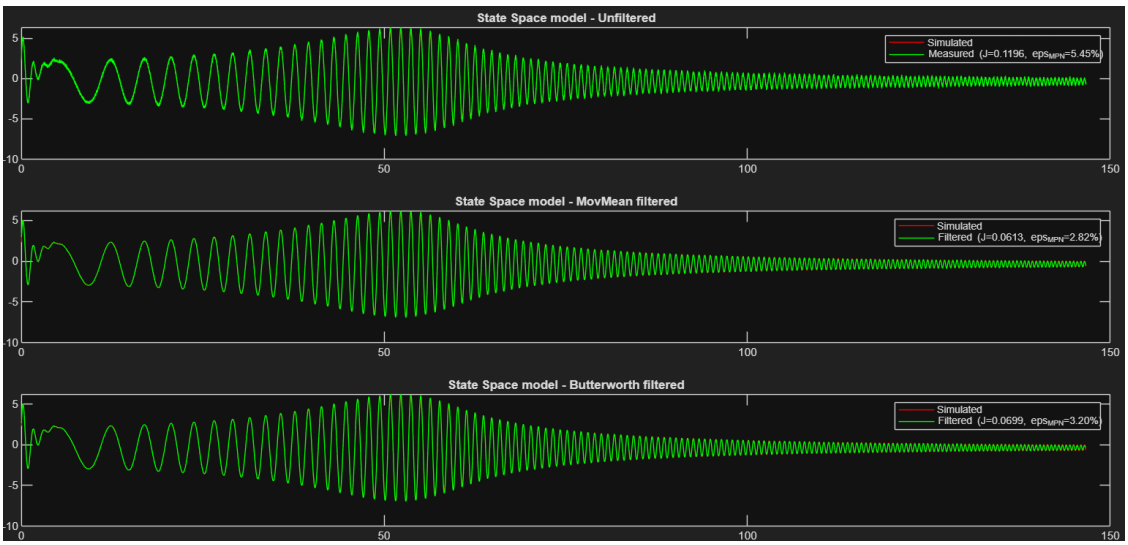


Figure 4 — Validation of the state-space model.

Transfer function model

The transfer function model has a much larger error, eps_MPN ≈ 14%, although it uses exactly the same parameters K, ζ and ωn. The difference comes from the initial conditions: in state space they were given explicitly ([y(1) 25]), whereas lsim applied to the transfer function starts from zero. That this is the reason, and not an unlucky choice of the data section, is shown by the fact that the large error no longer holds on test intervals not used for calibration.

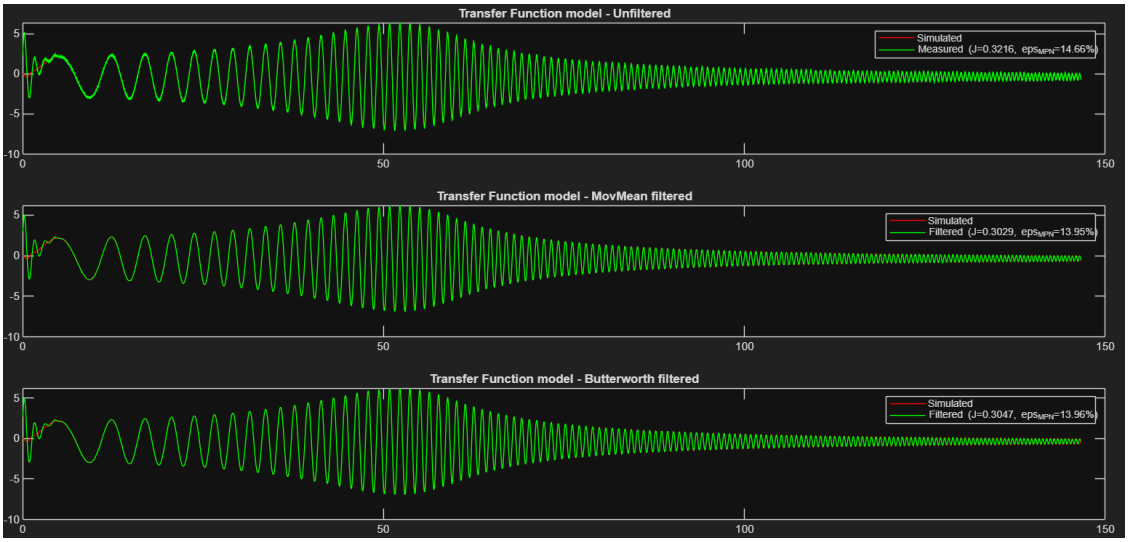


Figure 5 — Validation of the transfer function model.

Initially, the error of the signal was 11%, later reduced by fine adjustment of the indices. The lack of a good initial velocity launched the function far above the initial one, and a ζ chosen too large reduced the amplitude in the resonance region and amplified it outside it, which explained the large error. K also needed adjustments, because a single value affected by noise changed its value strongly.

Appendix — MATLAB source code

The appendix contains the complete MATLAB code used in this method: computing the constant K, determining ζ and ωn using the relations above, reconstructing the model (transfer function and state space), filtering the output and computing the performance indicators.

```
%% System identification
% Cur   s 5 - Modalitatea de calibrare a intrarii
% Curs 4 - Metodologie

i1 = 6083; % first clear max for y
i2 = 6178; % first clear max for u

K = y(i1) / u(i2);

i3 = 25421; % max rez y
i4 = 25782; % min rez y

i5 = 23800; % max u
i6 = 24187; % min u

Amply = (y(i3) - y(i4))/2;
Amplu = (u(i5) - u(i6))/2;
% 3 pct de maxim in jurul rezonantei => wk-1 < wr < wk

i7 = 24698; % first peak
i8 = 25383; % rez peak
i9 = 26140; % second peak

ToscPrev  = t(i8) - t(i7);
ToscAfter = t(i9) - t(i8);
Tosc = (ToscPrev + ToscAfter)/2;

wosc = 2*pi/Tosc;% =~ wrez
wr = wosc;
Mr = Amply / Amplu;


zeta = sqrt((1 - sqrt(1 - (K/Mr)^2) )/2);
wn = wr/ sqrt(1 - 2*zeta^2);

%% Transform
H = tf(K*wn^2, [1 2*zeta*wn wn^2]);
y_sim_tf = lsim(H,u,t);

%% State Space
A = [ 0 1;
   -wn.^2 -2*zeta*wn];
B = [0;
   K*wn.^2];
C = [1 0];
D = 0;

sys = ss(A,B,C,D);
y_sim_ss = lsim(sys,u,t, [y(1) 25]);

%% Filtering
idx_val = (i9+1):(length(t)- 1000); % Unused values

% Moving average filter
windowSize = 55;
y_mov = movmean(y,windowSize);

% Butterworth filter
fs = 1/Ts;
fc = 3;
ord = 2;
```

```
[b_filt, a_filt] = butter(ord, fc/(fs/2), 'low');
y_butter = filtfilt(b_filt, a_filt, y);
%% Performance indexes
% J = ||eps||_2 / sqrt(N)
% eps_MPN = ||eps||_2 / ||y - mean(y)||_2

% State Space - nefiltrat
J_ss_full    = norm(y_sim_ss - y) / sqrt(length(y));
eMPN_ss_full = norm(y_sim_ss - y) / norm(y - mean(y));
J_ss_val     = norm(y_sim_ss(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_ss_val  = norm(y_sim_ss(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% State Space - movmean
J_ss_mov_full    = norm(y_sim_ss - y_mov) / sqrt(length(y_mov));
eMPN_ss_mov_full = norm(y_sim_ss - y_mov) / norm(y_mov - mean(y_mov));
J_ss_mov_val     = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_ss_mov_val  = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% State Space - Butterworth
J_ss_butter_full    = norm(y_sim_ss - y_butter) / sqrt(length(y_butter));
eMPN_ss_butter_full = norm(y_sim_ss - y_butter) / norm(y_butter - mean(y_butter));
J_ss_butter_val     = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_ss_butter_val  = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

% Transfer Function - nefiltrat
J_tf_full    = norm(y_sim_tf - y) / sqrt(length(y));
eMPN_tf_full = norm(y_sim_tf - y) / norm(y - mean(y));
J_tf_val     = norm(y_sim_tf(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_tf_val  = norm(y_sim_tf(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% Transfer Function - movmean
J_tf_mov_full    = norm(y_sim_tf - y_mov) / sqrt(length(y_mov));
eMPN_tf_mov_full = norm(y_sim_tf - y_mov) / norm(y_mov - mean(y_mov));
J_tf_mov_val     = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_tf_mov_val  = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% Transfer Function - Butterworth
J_tf_butter_full    = norm(y_sim_tf - y_butter) / sqrt(length(y_butter));
eMPN_tf_butter_full = norm(y_sim_tf - y_butter) / norm(y_butter - mean(y_butter));
J_tf_butter_val     = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_tf_butter_val  = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

%% Plots - State Space model
figure
subplot(3,1,1)
plot(t,y_sim_ss,'r', t, y, 'g');
title('State Space model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_full, 100*eMPN_ss_full))

subplot(3,1,2)
plot(t,y_sim_ss,'r', t, y_mov, 'g');
title('State Space model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_mov_full, 100*eMPN_ss_mov_full))

subplot(3,1,3)
plot(t,y_sim_ss,'r', t, y_butter, 'g');
title('State Space model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_butter_full, 100*eMPN_ss_butter_full))

%% Plots - Transfer Function model
figure
subplot(3,1,1)
plot(t,y_sim_tf,'r', t, y, 'g');
title('Transfer Function model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_full, 100*eMPN_tf_full))

subplot(3,1,2)
plot(t,y_sim_tf,'r', t, y_mov, 'g');
title('Transfer Function model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_mov_full, 100*eMPN_tf_mov_full))
```

```
subplot(3,1,3)
plot(t,y_sim_tf,'r', t, y_butter, 'g');
title('Transfer Function model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_butter_full, 100*eMPN_tf_butter_full))
```